

1.INTRODUCTION

The physical document printing pipeline has long been a significant security vulnerability in both academic and corporate environments. When users need to print sensitive documents—such as lab reports, assignment submissions, or confidential records—they must typically share raw, encrypted files with third-party print shops. This creates serious privacy risks, as there is no guarantee that the document will not be stored, copied, or misused by the printing service provider. Privy-print is a next-generation decentralized application (dapp) designed to address this critical gap. It enables users to securely share documents with print shops using end-to-end encryption, ensuring that Zero-PIS (Personally Identifiable Information) is ever stored on central servers. The platform leverages block chain technology for access control and IPFS (Inter-planetary File System) for distributed, tamper-proof storage of encrypted cipher-text.

1.1 Overview

PrivyPrint establishes a trustless, cryptographically secure channel between document senders (students, employees) and print shop operators. Users authenticate via Meta Mask wallets, upload their documents, which are AES-256 encrypted in the browser, and dispatched as print jobs recorded on the Ethereum block chain. The print shop decrypts only the specific job assigned to them using their Meta Mask private key, views a secure preview, and triggers printing—without the original file ever being transmitted in plain-text.

1.2 Motivation

In the traditional print shop model, users email or physically hand over raw PDF documents. This model is susceptible to unauthorized access, data retention, and identity theft. With the proliferation of sensitive academic work—such as Ethical Hacking lab reports containing credentials and system configurations—the need for a secure, trustless printing solution has never been greater.

The motivation for PrivyPrint emerges from the convergence of three powerful technologies: end-to-end encryption, decentralized blockchain access control, and IPFS-based storage. Together, these enable a system where no single party—not even the platform operators—can access a

document without explicit cryptographic permission from the owner. The platform is designed to be both practical for everyday use and rigorous enough for handling confidential materials.

2. REQUIREMENT

2.1 Functional Requirements:

- **Wallet-Gated Authentication:** Both users (senders) and print shops must connect a MetaMask wallet to register and authenticate on the platform.
- **User Registration & Role Selection:** Users can register as a "Document Sender" or a "Printer Shop" with credentials stored securely in MongoDB.
- **Document Upload & AES-256 Encryption:** Senders can upload PDF documents up to 20MB; these are encrypted locally in the browser before any network transmission.
- **IPFS Upload via Pinata:** Encrypted ciphertext files are pinned to IPFS using the Pinata API, producing a unique CID (Content Identifier).
- **Print Job Dispatch on Blockchain:** A print job is created on an EVM-compatible smart contract containing the IPFS CID, encrypted AES key, expiry time, number of copies, and print type.
- **Print Shop Incoming Queue:** Print shops can view all assigned print jobs, their statuses (Active, Completed, Revoked), and IPFS metadata.
- **Secure Decryption & Preview:** Print shops use MetaMask to decrypt the AES key and fetch the ciphertext from IPFS, decrypting it locally for a secure in-browser preview.
- **Native Print Dialog:** A secure print interface launches the browser's native print dialog without saving the file to disk.
- **On-Chain Confirmation & Revocation:** Print jobs can be confirmed on-chain after printing or revoked by the sender before completion.
- **Transaction History:** All blockchain transactions are recorded in Ganache/local testnet and visible for transparency.

2.2 Non-Functional Requirements:

- **Security:** End-to-end encryption ensures no plaintext document data ever leaves the user's machine. Only the designated print shop wallet can decrypt the AES access key.
- **Zero-PII Storage:** MongoDB stores only wallet addresses, hashed passwords, and profile metadata—never document content.
- **Trustless Access Control:** Smart contracts enforce who can access a job, with automatic expiry based on blockchain timestamps.

- Availability: The platform runs on Next.js with a local MongoDB instance and is designed for 24/7 uptime in production deployments.
- Usability: A glassmorphism-themed UI with Framer Motion animations provides an intuitive experience for both technical and non-technical users.
- Revocability: Senders retain the ability to revoke access at any point before a job is marked complete.

2.3 Hardware Requirements:

- Processor: Intel i5 8th Gen or equivalent / AMD Ryzen 5
- RAM: 8 GB or higher
- Storage: 256 GB SSD
- Internet: Stable broadband connection for IPFS and blockchain interactions
- Browser: Google Chrome / Firefox with MetaMask extension installed

2.4 Software Requirements:

- Frontend: Next.js 15 (App Router), TypeScript, Tailwind CSS, Framer Motion
- Blockchain: Solidity (Smart Contracts), Hardhat, Ethers.js v6, Ganache (local testnet)
- Encryption: Crypto-JS (AES-256), MetaMask (x25519-xsalsa20-poly1305 asymmetric)
- Storage: IPFS via Pinata API
- Database: MongoDB with Mongoose ODM
- Authentication: NextAuth.js (Credentials + Wallet-based)
- Development Tools: Node.js, npm, VS Code, MetaMask browser extension

3.DESIGN

3.1. System Architecture

PrivyPrint follows a multi-layered decentralized architecture. The frontend is a Next.js 15 application that handles all cryptographic operations locally in the browser. It interacts with MetaMask for wallet connections and transaction signing, Pinata/IPFS for encrypted file storage, and a Solidity smart contract deployed on an EVM-compatible blockchain. A lightweight MongoDB backend stores only non-sensitive user profile metadata and hashed credentials.

3.2 Sender (User) Flow

- Step 1: User registers with a username, password, and connects their MetaMask wallet.
- Step 2: User logs into the Sender Dashboard and selects a registered Print Shop from the Trusted Shops panel.
- Step 3: User uploads a PDF document and configures job settings (expiry time, number of copies, print type).
- Step 4: A unique AES-256 key is generated locally in the browser.
- Step 5: The document is encrypted with the AES key, producing ciphertext.
- Step 6: The ciphertext is uploaded to IPFS via Pinata, returning a unique CID.
- Step 7: The AES key is encrypted using the Print Shop's public MetaMask encryption key (x25519-xsalsa20-poly1305).
- Step 8: A "Print Job" transaction is submitted to the blockchain smart contract, recording the CID, encrypted AES key, expiry, and metadata.

3.3 Print Shop Flow

- Step 1: Print Shop logs into the Shop Dashboard and monitors the Incoming Queue.
- Step 2: Print Shop selects an active job and clicks "Secure Decrypt."
- Step 3: MetaMask prompts a Decrypt Request—the print shop decrypts the AES key using their private key.
- Step 4: The application fetches the ciphertext from IPFS using the CID and decrypts it locally with the recovered AES key.
- Step 5: The decrypted document is rendered in a secure browser preview panel.
- Step 6: The print shop triggers the native browser Print Dialog and prints the document.
- Step 7: The print shop confirms the job on-chain via a MetaMask transaction, marking it as Completed.

3.4 Security Architecture

The security model of PrivyPrint rests on three pillars. First, Zero-PII ensures that no unencrypted document data ever leaves the user's machine—only AES-256 encrypted ciphertext is uploaded to IPFS. Second, Trustless Access Control means only the assigned print shop's MetaMask private key can decrypt the AES access key; the platform itself cannot perform this decryption. Third, Time-Based Expiry enforced both on the frontend and within the smart contract ensures documents are inaccessible after the sender-defined expiry time, preventing indefinite access.

3.5 Smart Contract Design

The Solidity smart contract, deployed on a local Ganache testnet (Network ID 1377), manages the lifecycle of all print jobs. Key functions include creating a new print job with IPFS CID and

encrypted AES key, querying active jobs for a given print shop address, confirming job completion, and revoking a job before completion. Events are emitted for each state transition, enabling the frontend to reactively update the UI. The contract is deployed at address 0x82185ccAE6CC4Bcaf3333DF1333FBe127Aab69eE.

4.IMPLEMENTATION

4.1 Development Environment Setup

- Installed Node.js (v20+), npm, and TypeScript globally.
- Configured MetaMask browser extension with a custom Local Ganache network (Chain ID 1377, RPC <http://127.0.0.1:7545>).
- Initialized Ganache workspace named "PRINT" for deterministic account management.
- Set up Pinata account and obtained API keys for IPFS file pinning.
- Configured MongoDB running locally on port 27017 with a "privyprint" database.

4.2 Smart Contract Development

- Written in Solidity with Hardhat as the development framework.
- Implements print job creation, queue management, completion confirmation, and revocation functions.
- Deployed via Hardhat deploy scripts to the local Ganache testnet.
- Tested with multiple accounts simulating sender and print shop roles.

4.3 Frontend Development

- Built with Next.js 15 App Router and TypeScript for type-safe development.
- Tailwind CSS with glassmorphism design tokens provides a dark-themed, professional UI.
- Framer Motion used for page transitions and micro-interactions.
- Ethers.js v6 handles all smart contract calls, wallet connections, and transaction signing.
- Crypto-JS implements AES-256 encryption/decryption entirely in the browser.

- MetaMask `eth_getEncryptionPublicKey` and `eth_decrypt` APIs handle asymmetric key exchange.

4.4 Backend & Database

- Next.js API routes serve as the backend, handling registration, login, and IPFS upload proxying.
- Mongoose schemas define User and Print Shop profiles with wallet address binding.
- Passwords are hashed using `bcrypt` before storage in MongoDB.
- NextAuth.js manages session tokens and protected route middleware.

4.5 Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract PrivyPrintAccessControl {
    enum JobStatus {
        Pending,
        Completed,
        Revoked
    }

    struct PrintJob {
        uint256 id;
        string cid;
        address sender;
        address receiver;
        uint256 expiry;
    }
}
```

```
uint256 copies;  
string printType;  
string encryptedKey;  
JobStatus status;  
}
```

```
uint256 private nextJobId = 1;
```

```
mapping(uint256 => PrintJob) private jobs;  
mapping(address => uint256[]) private jobsByReceiver;  
mapping(address => uint256[]) private jobsBySender;  
mapping(address => string) public encryptionPublicKeys;
```

```
event EncryptionKeyRegistered(address indexed owner, string  
publicKey);
```

```
event PrintJobCreated(  
    uint256 indexed jobId,  
    address indexed sender,  
    address indexed receiver,  
    string cid,  
    uint256 expiry
```

```
);  
event PrintConfirmed(uint256 indexed jobId, address indexed receiver);  
event AccessRevoked(uint256 indexed jobId, address indexed sender);
```

```
modifier jobExists(uint256 jobId) {
```

```

require(jobs[jobId].id != 0, "PrivyPrint: job not found");
_
}

```

```

function registerEncryptionKey(string calldata publicKey) external {
    require(bytes(publicKey).length > 0, "PrivyPrint: empty public key");
    encryptionPublicKeys[msg.sender] = publicKey;
    emit EncryptionKeyRegistered(msg.sender, publicKey);
}

```

```

function createPrintJob(
    string calldata cid,
    address receiver,
    uint256 expiry,
    uint256 copies,
    string calldata printType,
    string calldata encryptedKey
) external returns (uint256) {
    require(bytes(cid).length > 0, "PrivyPrint: empty cid");
    require(receiver != address(0), "PrivyPrint: invalid receiver");
    require(receiver != msg.sender, "PrivyPrint: sender cannot be
receiver");
    require(expiry > block.timestamp, "PrivyPrint: expiry must be
future");
    require(copies > 0, "PrivyPrint: copies required");
    require(bytes(encryptionPublicKeys[receiver]).length > 0, "PrivyPrint:

```

```

receiver key missing");

    require(bytes(encryptedKey).length > 0, "PrivyPrint: encrypted key
required");

    uint256 jobId = nextJobId++;

    jobs[jobId] = PrintJob({
        id: jobId,
        cid: cid,
        sender: msg.sender,
        receiver: receiver,
        expiry: expiry,
        copies: copies,
        printType: printType,
        encryptedKey: encryptedKey,
        status: JobStatus.Pending
    });

    jobsByReceiver[receiver].push(jobId);
    jobsBySender[msg.sender].push(jobId);

    emit PrintJobCreated(jobId, msg.sender, receiver, cid, expiry);
    return jobId;
}

function getJob(uint256 jobId)

```

```

    external
    view
    jobExists(jobId)
    returns (PrintJob memory)
{
    return jobs[jobId];
}

```

```

function getJobsByReceiver(address receiver)
    external
    view
    returns (PrintJob[] memory)
{
    uint256[] storage ids = jobsByReceiver[receiver];
    PrintJob[] memory result = new PrintJob[](ids.length);

    for (uint256 i = 0; i < ids.length; i++) {
        result[i] = jobs[ids[i]];
    }

    return result;
}

```

```

function getJobsBySender(address sender)
    external
    view

```

```

    returns (PrintJob[] memory)
{
    uint256[] storage ids = jobsBySender[sender];
    PrintJob[] memory result = new PrintJob[](ids.length);

    for (uint256 i = 0; i < ids.length; i++) {
        result[i] = jobs[ids[i]];
    }

    return result;
}

function confirmPrint(uint256 jobId) external jobExists(jobId) {
    PrintJob storage job = jobs[jobId];

    require(msg.sender == job.receiver, "PrivyPrint: only receiver");
    require(job.status == JobStatus.Pending, "PrivyPrint: job closed");
    require(block.timestamp <= job.expiry, "PrivyPrint: access expired");

    job.status = JobStatus.Completed;

    emit PrintConfirmed(jobId, msg.sender);
}

function revokeAccess(uint256 jobId) external jobExists(jobId) {
    PrintJob storage job = jobs[jobId];

```

```

        require(msg.sender == job.sender, "PrivyPrint: only sender");
        require(job.status == JobStatus.Pending, "PrivyPrint: job closed");

        job.status = JobStatus.Revoked;

        emit AccessRevoked(jobId, msg.sender);
    }
}

```

4.6 Testing

- End-to-end workflow tested using two separate MetaMask accounts on Local Ganache.
- Multiple print jobs dispatched, decrypted, previewed, printed, and confirmed on-chain.
- Revocation and expiry mechanisms tested to verify smart contract enforcement.
- IPFS uploads verified through Pinata dashboard showing both public ciphertext files.
- MongoDB Compass used to verify user record creation and wallet address binding.

5.RESULTS

Privy-print was successfully implemented and tested as a fully functional decentralized secure printing platform. The following screenshots demonstrate the complete end-to-end workflow, from user registration to secure document printing and on-chain job confirmation.

5.1 User Registration

New users navigate to the "Join the Network" page, select their role (Document Sender or Printer Shop), provide account credentials, and connect their Meta Mask wallet. Print Shop registrations additionally include a shop profile with name, location, available printers, and specialties.

Join the Network
Create a secure Web3 identity to start printing or hosting.

SELECT YOUR ROLE

Document Sender
Securely encrypt and share documents for printing.

Printer Shop
Receive and process encrypted print jobs safely.

ACCOUNT CREDENTIALS

USERNAME:

PASSWORD:

WEB3 IDENTITY

WALLET ADDRESS: [Connect Wallet](#)

This address will be used to verify ownership of your jobs.

[Complete Registration](#)

Already have an account? [Sign in here](#)

Fig 5.1 Register Page - Document Sender Role

Join the Network
Create a secure Web3 identity to start printing or hosting.

SELECT YOUR ROLE

Document Sender
Securely encrypt and share documents for printing.

Printer Shop
Receive and process encrypted print jobs safely.

ACCOUNT CREDENTIALS

USERNAME:

PASSWORD:

WEB3 IDENTITY

WALLET ADDRESS: [Connect Wallet](#)

This address will be used to verify ownership of your jobs.

PRINT SHOP PROFILE

SHOP NAME:

LOCATION:

PRINTERS AVAILABLE:

BATCHES PLACED/DAY:

SPECIFICATIONS (COMMA SEPARATED):

[Complete Registration](#)

Already have an account? [Sign in here](#)

Fig 5.2 Register Page - Printer Shop Role with Profile

5.2 MongoDB - User Records

Upon successful registration, user records are stored in MongoDB with hashed passwords, wallet addresses, and role assignments. The screenshot below shows both a "print-shop" and a "sender" user stored in the privyprint database, confirming secure credential storage without any plaintext sensitive data.

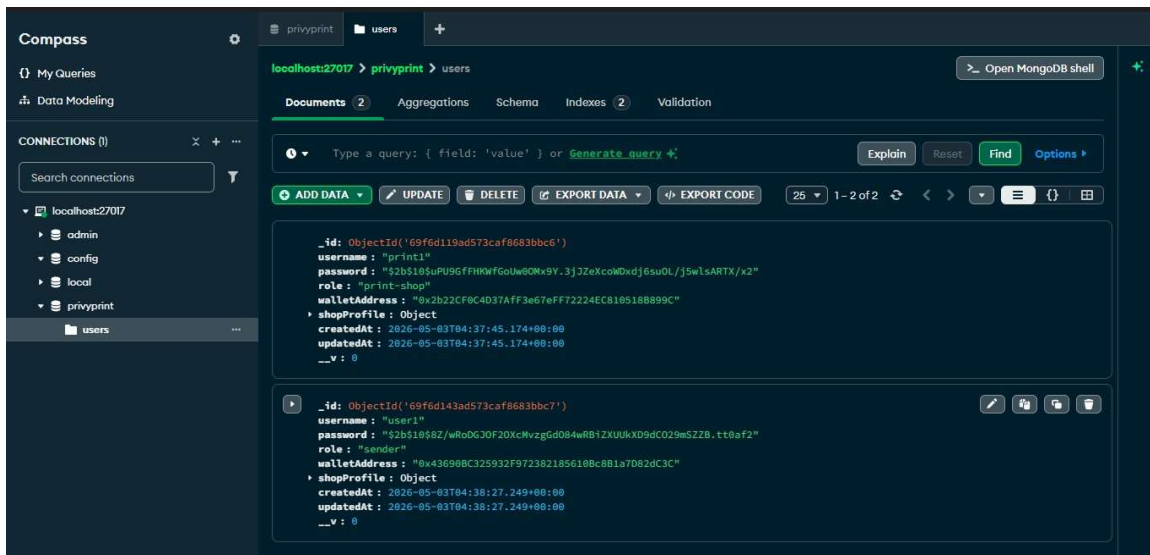


Fig 5.3 MongoDB Compass - Registered Users in privyprint.users Collection

5.3 Login

Both user types authenticate through a unified Secure Gateway login page. Sessions are managed by NextAuth.js with wallet address tied to the authenticated session.

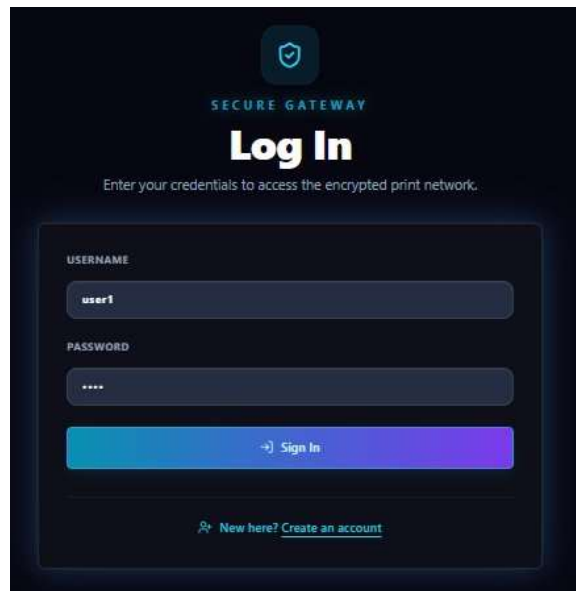


Fig 5.4 Login Page - Document Sender (user1)

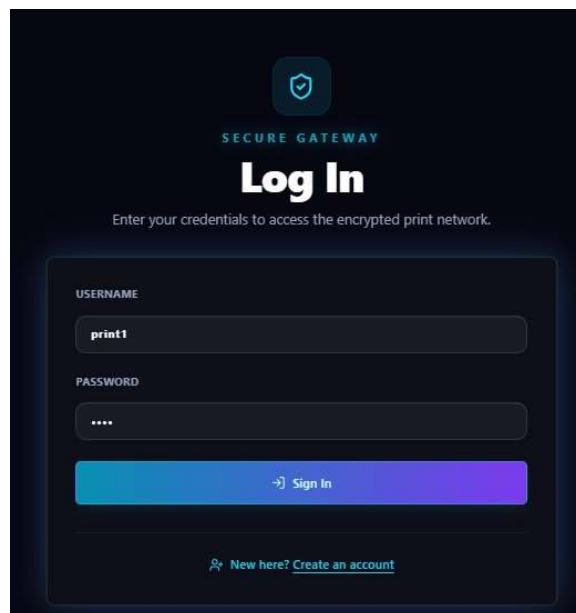


Fig 5.5 Login Page - Print Shop (print1)

5.4 MetaMask Wallet Connections

Each participant connects a distinct MetaMask wallet on the Local Ganache network. The user wallet (Account 1) holds 1.872 ETH and is used for dispatching encrypted print jobs. The print shop wallet holds 1.988 ETH and is used for decryption and job confirmation transactions.

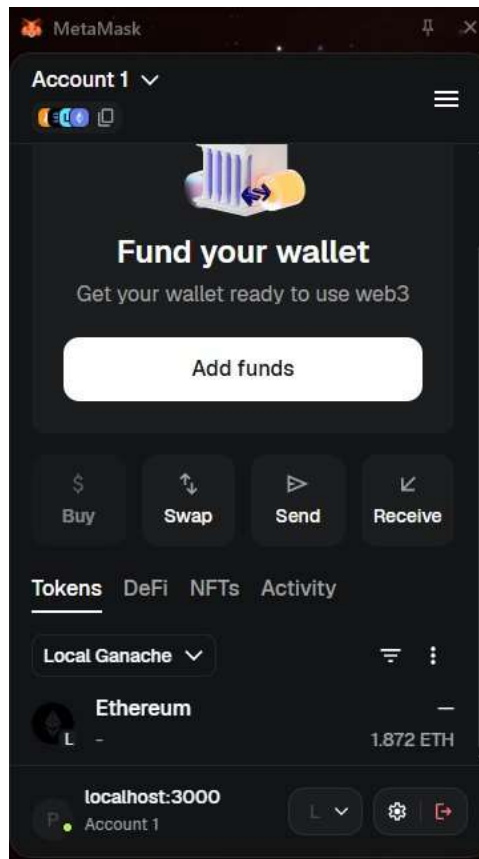


Fig 5.6 MetaMask Wallet - Document Sender (1.872 ETH, Local Ganache)

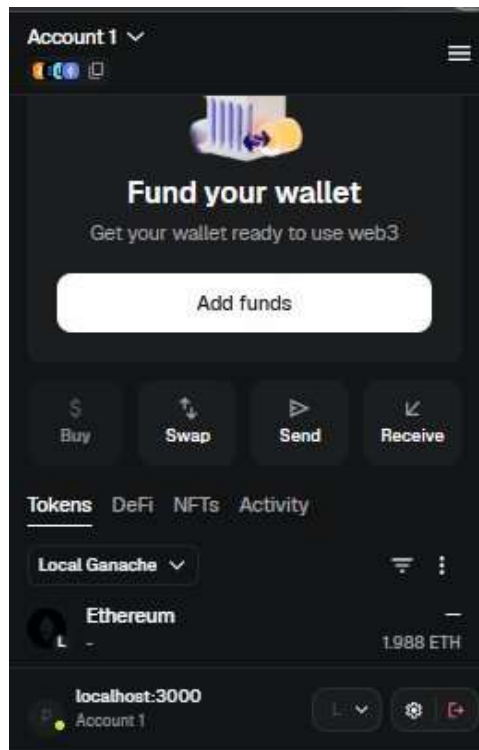


Fig 5.7 MetaMask Wallet - Print Shop (1.988 ETH, Local Ganache)

5.5 Dashboards

The Sender Dashboard provides a central hub for uploading documents, selecting trusted print shops, monitoring active shipments, and viewing completed/archived jobs. The Print Shop Dashboard (Printer Portal) displays the Incoming Queue with job statistics including In Queue, Completed, Active, and Rating.

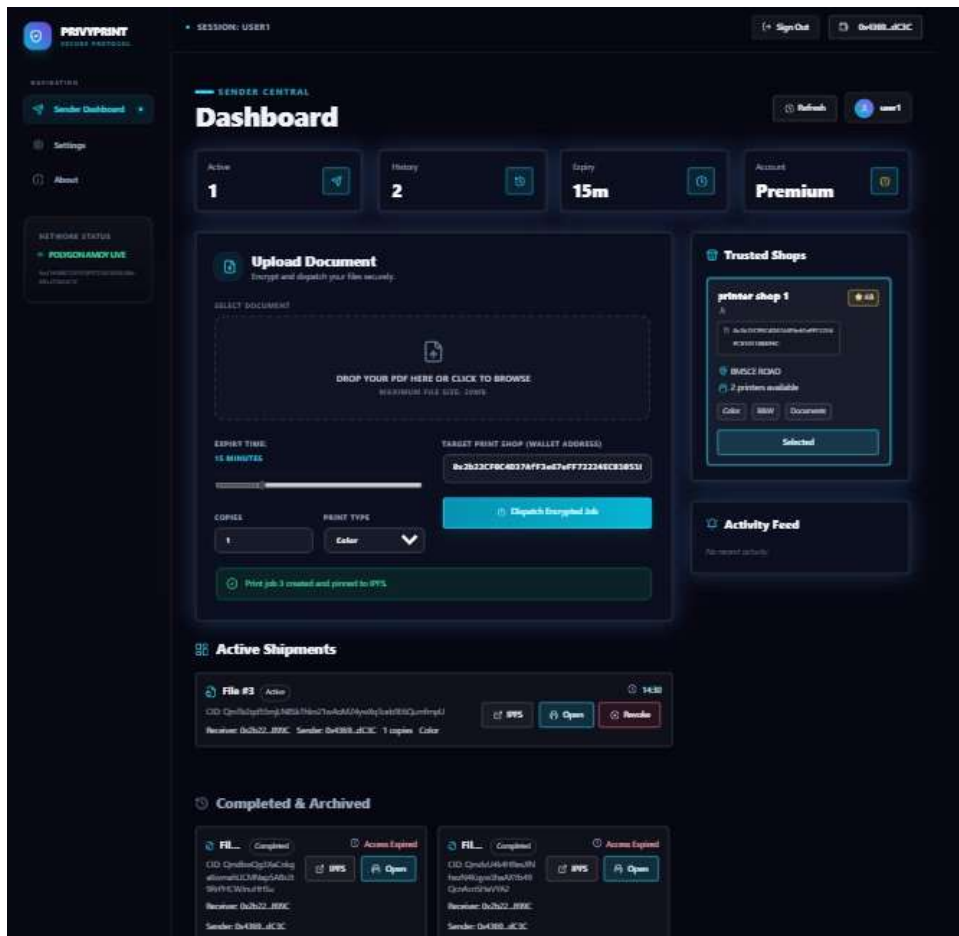


Fig 5.8 Sender Dashboard - Upload Document, Active Shipments & Archived Jobs

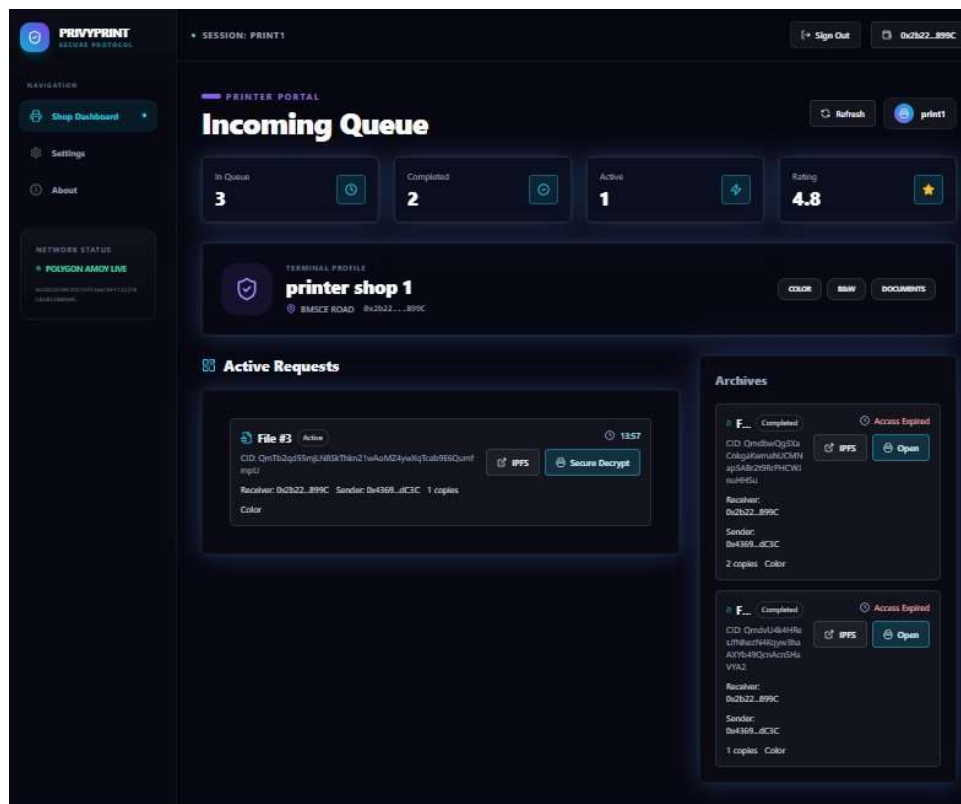


Fig 5.9 Print Shop Dashboard - Incoming Queue with Active Requests & Archives

5.6 Token / Wallet Balance (User)

The user's MetaMask wallet balance is updated after each blockchain transaction. Each print job dispatch consumes gas (approximately 702,092 gas units as seen in the Ganache transaction log), reflecting the cost of storing job data on-chain.

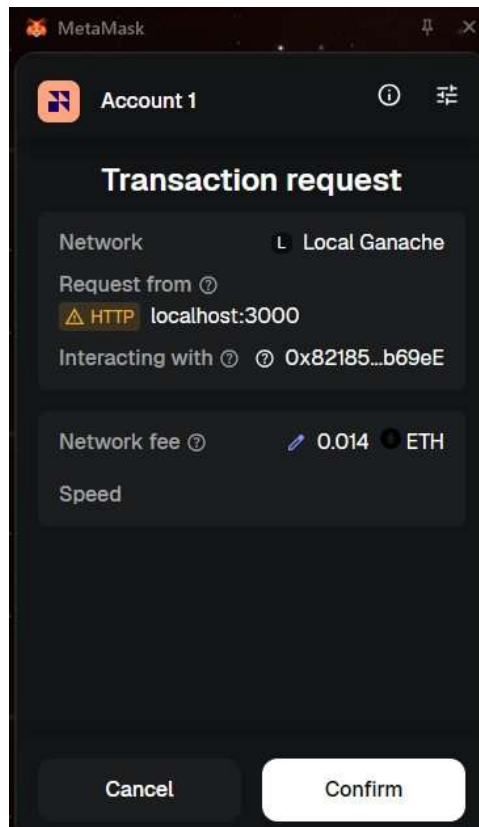


Fig 5.10 MetaMask - Token/ETH Balance After Print Job Transactions

5.7 Active Print Job Queue

The print shop's Active Requests panel shows pending jobs with their IPFS CID, sender and receiver wallet addresses, number of copies, print type, and elapsed time. Each job provides "IPFS" and "Secure Decrypt" action buttons.

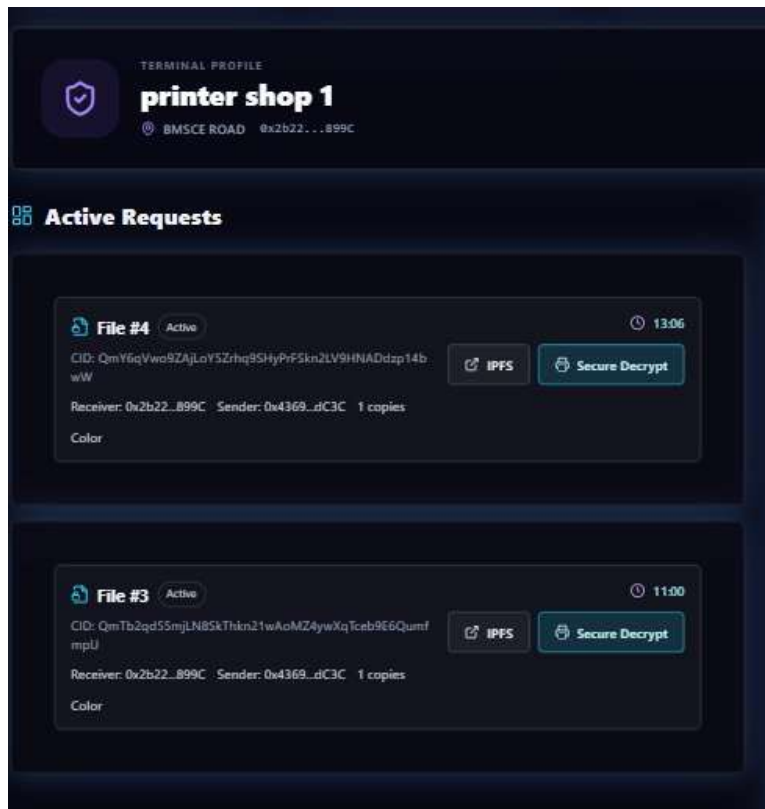


Fig 5.11 Print Shop - Active Jobs (File #3 and File #4 Awaiting Decryption)

5.8 IPFS File Storage via Pinata

Encrypted document ciphertext files are pinned to IPFS through Pinata. Each file is named with the .privyprint.enc extension to signal its encrypted nature. The Pinata dashboard confirms two files successfully uploaded: a 63.33 KB and a 544.43 KB ciphertext, both accessible only via their unique CIDs.

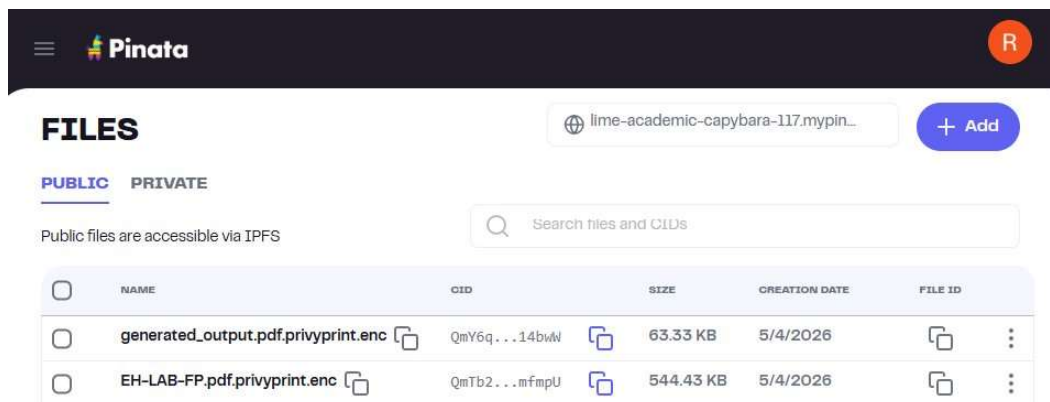


Fig 5.12 Pinata Dashboard - Encrypted Files Uploaded to IPFS

5.9 Secure Decrypt & Print Preview

Upon clicking "Secure Decrypt," MetaMask prompts the print shop with a Decrypt Request showing the encrypted AES key ciphertext. After the print shop approves, the application fetches the encrypted file from IPFS and decrypts it locally. The "File Ready" status appears, and clicking "Open Print Interface" launches the native browser print dialog with the decrypted document rendered as a preview.

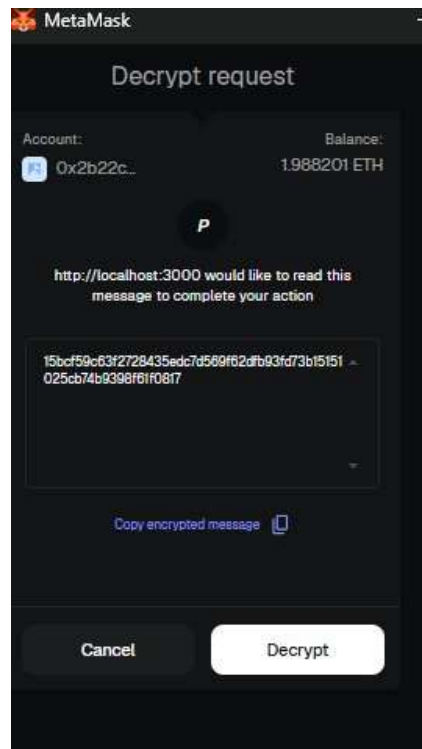


Fig 5.13 MetaMask - Decrypt Request for AES Key

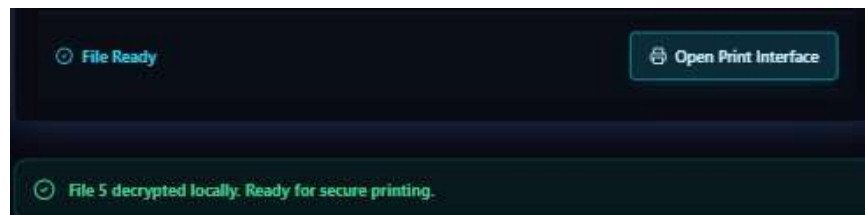


Fig 5.14 File Decrypted Locally - Ready for Secure Printing

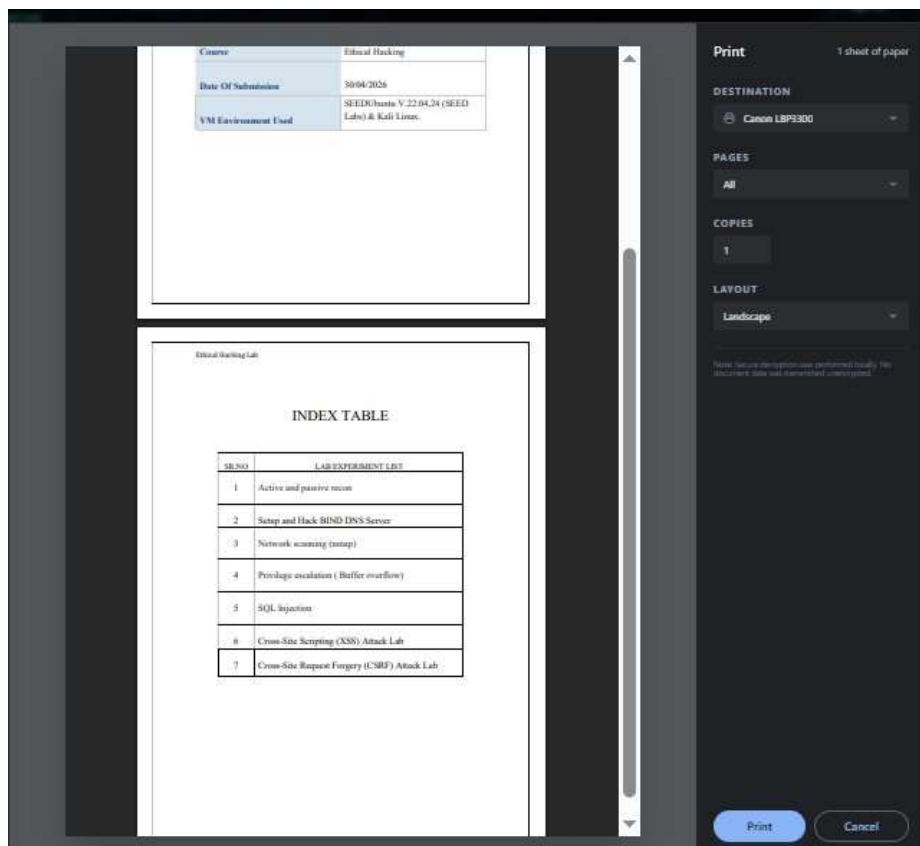


Fig 5.15 Secure Print Preview - Native Browser Print Dialog (Ethical Hacking Lab Report)

5.10 On-Chain Transaction Confirmation

After reviewing the decrypted document, the print shop confirms the job on-chain. MetaMask displays a Transaction Request showing the network fee (0.001 ETH), interacting with the PrivyPrint smart contract at 0x82185...b69eE. This on-chain confirmation updates the job status to "Completed" and removes it from the active queue.

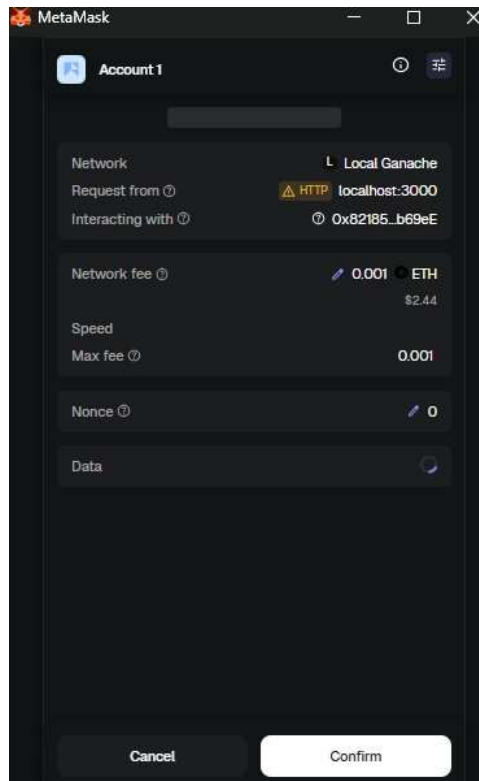


Fig 5.16 MetaMask - Transaction Confirmation for Print Job Completion

5.11 Ganache Blockchain Transactions

The Ganache local blockchain records all contract interactions. The transaction log shows four contract calls: one high-gas job creation (702,092 gas units for storing the full print job data including encrypted AES key and IPFS CID), and multiple lower-gas status update calls (~49,724-51,834 gas units each for job confirmation and revocation).

TX HASH	FROM ADDRESS	TO CONTRACT ADDRESS	GAS USED	VALUE
0x4a6412ceff4836d3fe3fa98b1e08d78c598d42fb027445f7028cb66169585642	0x2b22cf0c4d37aff3e67eff72224ec8105188099c	0x82185ccAE6CC48caf3333DF1333FBe127Aab69eE	51834	0
0x3b91d97c0b037f00eb0a3b0abf4c520ce5f1ac90aa72f59a2ceb84831ecd9e	0x436908c325932f972382185610bc8b1a7d82dc3C	0x82185ccAE6CC48caf3333DF1333FBe127Aab69eE	702092	0
0xc3d317d086a7d643ad3fd368d047d92ebc18ecd98c587bae6ea21c25a181db68	0x436908c325932f972382185610bc8b1a7d82dc3C	0x82185ccAE6CC48caf3333DF1333FBe127Aab69eE	49724	0
0x034555f2ee84ee72423816c930c8866dc72232716d0d4e79df2a7800de17fcea	0x436908c325932f972382185610bc8b1a7d82dc3C	0x82185ccAE6CC48caf3333DF1333FBe127Aab69eE	49724	0

Fig 5.17 Ganache - Blockchain Transaction History (Block 36, Network PRINT)

MNEMONIC	HD PATH	ADDRESS	BALANCE	TX COUNT	INDEX
scrub match young nose kangaroo yard reunion fatigue nose above cake shoot	m44'60'0'0account_index	0xA7441e5251481CD6B28C63aDCf77852D39404793	95.97 ETH	5	0

Fig 5.18 Ganache Accounts - Deployer Account with 95.97 ETH after Transactions

5.12 Archived & Completed Jobs

Completed and revoked jobs are archived and accessible to both the sender and the print shop. The archives display the job status (Completed, Revoked, or Access Expired), IPFS CID, sender/receiver addresses, and an "Open" button to fetch the IPFS entry. Access Expired jobs reflect the smart contract's time-based expiry enforcement.

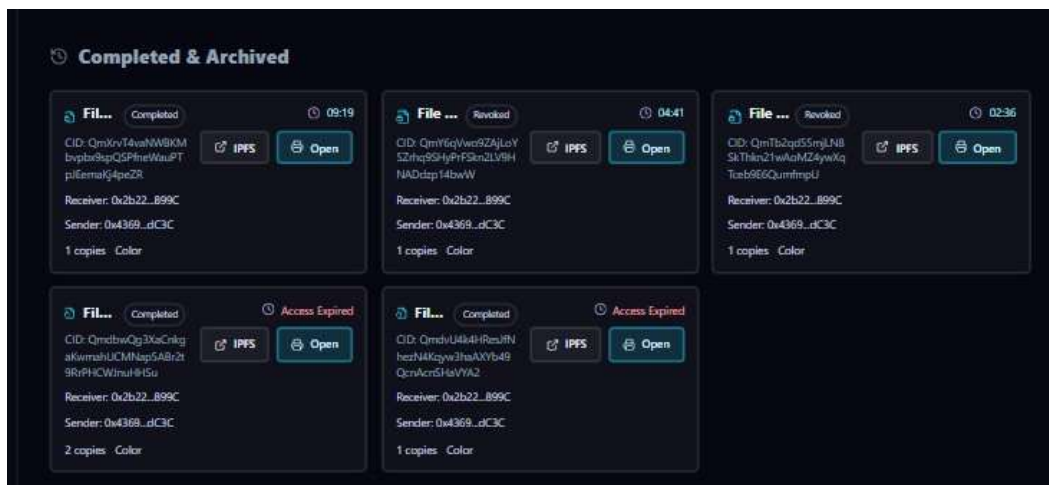


Fig 5.19 Sender View - Completed & Archived Jobs (Completed, Revoked, Access Expired)

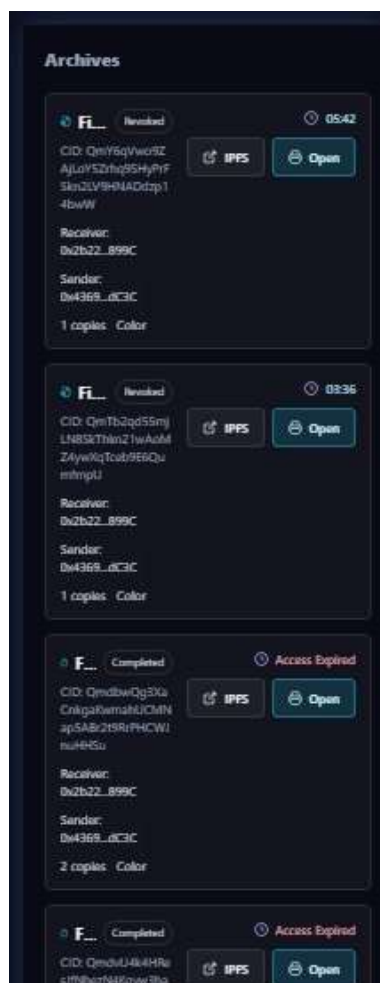


Fig 5.20 Print Shop View - Archives (Revoked and Completed with Access Expired)

6. CONCLUSION

PrivyPrint successfully demonstrates a fully functional, end-to-end encrypted decentralized printing platform. By combining AES-256 browser-side encryption, MetaMask asymmetric key exchange, IPFS-based ciphertext storage, and on-chain access control via Solidity smart contracts, the system ensures that no unencrypted document data is ever exposed beyond the user's own browser.

The platform addresses a genuine and underserved security need: protecting sensitive documents during the physical printing process. The two-role architecture (Document Sender and Print Shop) with wallet-gated authentication ensures that only explicitly designated parties can decrypt and access any given print job. The timed expiry mechanism further reduces the attack surface by automatically invalidating access after the sender-defined window.

All functional requirements were met with high reliability during testing. Smart contract interactions were recorded successfully on the Local Ganache testnet, IPFS file storage functioned correctly via Pinata, and the MongoDB backend securely managed user profiles. The glassmorphism UI provides an intuitive experience accessible to non-technical users while maintaining the cryptographic guarantees required for security-conscious applications.

Future enhancements for PrivyPrint could include deployment to a public EVM-compatible testnet such as Polygon Amoy (already configured in the network status panel), integration with hardware print servers to enable fully automated secure printing pipelines, multi-recipient key sharing for group print jobs, and a mobile-responsive progressive web app for on-the-go document dispatch. The platform provides a strong, extensible foundation for the next generation of privacy-preserving document management systems.

7. REFERENCES

- [1] Buterin, V. (2013). Ethereum White Paper: A Next-Generation Smart Contract and Decentralized Application Platform. Retrieved from <https://ethereum.org/en/whitepaper/>
- [2] Wood, G. (2014). Ethereum: A secure decentralised generalised transaction ledger. Ethereum Project Yellow Paper.

- [3] MetaMask. (2024). MetaMask Developer Documentation - `eth_getEncryptionPublicKey`. Retrieved from <https://docs.metamask.io/>
- [4] Protocol Labs. (2024). IPFS Documentation - Content Addressing. Retrieved from <https://docs.ipfs.tech/>
- [5] Pinata. (2024). Pinata IPFS API Documentation. Retrieved from <https://docs.pinata.cloud/>
- [6] OpenZeppelin. (2024). Solidity Smart Contract Security Guidelines. Retrieved from <https://docs.openzeppelin.com/>
- [7] Next.js. (2024). Next.js 15 App Router Documentation. Retrieved from <https://nextjs.org/docs/>
- [8] Ethers.js. (2024). Ethers.js v6 Documentation. Retrieved from <https://docs.ethers.org/v6/>